OOOP WITH JAVA IMP QUE ANS ALL UNIT

UNIT 1

FAQ(s):

Q1. Explain the features of Java.

Java is a powerful, object-oriented, platform-independent programming language developed by Sun Microsystems. It became popular because it solves the problems of C and C++ and provides a secure and portable environment for application development. Java is widely used for web applications, enterprise software, mobile apps (Android), embedded systems, and cloud applications.
Its popularity and wide usage are the result of the following features:

1. Simple
Java syntax is clean and easy to learn because it is similar to C and C++.
Complex features like
pointers,
operator overloading,
multiple inheritance through classes, and
manual memory management
are removed to reduce programmer errors.

2. Object-Oriented
Java is a pure object-oriented language because everything is written in the form of classes and objects.

It supports all OOP principles:
Encapsulation: wrapping data and methods into a class.
Inheritance: reusing properties of existing classes.
Polymorphism: using the same method in different ways.
Abstraction: hiding internal details and showing only essential features.

This increases code reusability and maintainability.

3. Platform Independent
Java's most important feature is Write Once, Run Anywhere (WORA).
Java compiler generates bytecode (.class file) instead of machine code.
The bytecode can run on any operating system (Windows, Linux, Mac) that has JVM installed.
This makes Java highly portable and flexible.

4. Secure
Java provides strong security at multiple levels.
It does not support pointers which prevent unauthorized memory access.
The bytecode verifier checks for illegal code.

Java also supports a Security Manager, which controls file and network access.
Hence, Java is widely used in banking, e-commerce, and enterprise applications.

5. Robust
Robust means strong and error-free.
Java provides:

Strong type checking
Exception handling
Automatic garbage collection
Runtime error detection

This prevents program crashes and improves reliability.

6. Multithreaded

Java allows multiple tasks to run simultaneously using threads.
This improves performance in applications like:

animations
gaming
background services
video streaming

7. High Performance
Java uses a Just-In-Time (JIT) compiler, which converts bytecode into machine code at runtime, making execution faster.

8. Distributed
Java supports distributed applications through libraries such as:

RMI (Remote Method Invocation)
JDBC
Socket programming
This helps in building network-based applications.

9. Dynamic
Java can load classes dynamically at runtime based on conditions.
This makes applications flexible and easily expandable.

✔ Conclusion

Java is simple, secure, portable, object-oriented, robust, and suitable for modern applications.
These features make Java one of the most widely used programming languages worldwide.

Q2. Define JVM / Bytecode and explain its working.

Java uses a unique architecture where the program is not directly converted into machine code. Instead, it uses an intermediate format called bytecode, and a runtime environment called JVM for execution.

Java Virtual Machine (JVM)

JVM is a software-based engine that runs Java bytecode.
It performs:

loading of code
verifying code
converting bytecode to machine code
memory management
garbage collection
ensuring security

JVM is platform-dependent, but bytecode is platform-independent.

Bytecode (.class file)
Bytecode is a special kind of instruction set generated by the Java compiler.
It is not machine code; it is universal code that can run on any device with JVM.
This enables Java's Write Once, Run Anywhere capability.

Working of JVM (Step-by-Step)

1. Writing Source Code
Programmer writes a Java program in a .java file.

2. Compilation (javac compiler)

When we compile the file:
javac Test.java
The compiler checks the syntax and generates bytecode in the form of .class file.

3. Class Loader

JVM uses Class Loader to:
Load classes into memory
Check class dependencies
Manage memory for class files

4. Bytecode Verifier

This part checks the bytecode for:
illegal code
invalid memory access
code violating security rules
If code is unsafe, execution stops.

5. Interpreter and JIT Compiler

JVM has two execution engines:
✔ Interpreter
Executes bytecode line-by-line.
✔ JIT (Just-In-Time) Compiler
Converts repeated bytecode into native machine code for faster execution.

6. Runtime Execution
Finally, JVM executes the machine code on the processor.

✔ Conclusion

JVM and bytecode make Java portable, secure, and efficient.
This architecture separates compilation from execution, giving Java a unique advantage over other languages.

Q3. Programs based on loops (Fibonacci, Reverse number, Palindrome, Max of 3 numbers).
Below are all required loop-based programs in clean, exam-friendly format.

A. Fibonacci Series

```
class Fibonacci {
    public static void main(String[] args) {
        int a = 0, b = 1, c;
        System.out.print(a + " " + b + " ");
        for (int i = 2; i < 10; i++) {
            c = a + b;
            System.out.print(c + " ");
            a = b;
            b = c;
        }
    }
}
```

B. Reverse Number

```
class Reverse {
    public static void main(String[] args) {
        int n = 1234, rev = 0;
        while (n != 0) {
            rev = rev * 10 + n % 10;
            n = n / 10;
        }
        System.out.println("Reversed Number = " + rev);
    }
}
```

C. Palindrome Number

```
class Palindrome {
    public static void main(String[] args) {
        int n = 121, temp = n, rev = 0;

        while (n != 0) {
            rev = rev * 10 + n % 10;
            n /= 10;
        }
        if (temp == rev)
            System.out.println("Palindrome Number");
        else
            System.out.println("Not a Palindrome");
    }
}
```

D. Maximum of Three Numbers

```
class MaxOfThree {
    public static void main(String[] args) {
        int a = 40, b = 10, c = 25;

        int max = (a > b && a > c) ? a : (b > c ? b : c);

        System.out.println("Maximum = " + max);
    }
}
```

Q4. Demonstrate String class methods: charAt(), contains(), format(), length(), split().

Below is a program demonstrating all required methods.

String Methods Demonstration

```
class StringMethods {
    public static void main(String[] args) {
        String s = "Hello World";

        // charAt()
        System.out.println("charAt(1): " + s.charAt(1));

        // contains()
        System.out.println("contains(\"World\"): " + s.contains("World"));

        // format()
        System.out.println("format(): " + String.format("Value = %d", 50));

        // length()
        System.out.println("length(): " + s.length());

        // split()
        String[] parts = s.split(" ");
        System.out.println("split(): ");
        for (String p : parts) {
            System.out.println(p);
        }
    }
}
```

Repeated Questions:

Q1. Reverse a Number – Explain with Program

Reversing a number means obtaining a new number whose digits appear in the opposite order compared to the original number.
For example:
Input: 1234
Output: 4321

Concept Explanation:

A number can be reversed using the following steps:

1. Extract the last digit of the number using n % 10
2. Add this digit to the reverse number
3. Remove the last digit from the original number using n / 10
4. Repeat the process until the number becomes 0

The reverse is constructed as:
reverse = reverse * 10 + digit

Java Program to Reverse a Number

```
public class ReverseNumber {
    public static void main(String[] args) {

        int num = 1234;
        int rev = 0;
        int temp = num;

        while (temp != 0) {
            int digit = temp % 10;       // extract last digit
            rev = rev * 10 + digit;      // append digit to reverse
            temp = temp / 10;            // remove last digit
        }

        System.out.println("Original Number: " + num);
        System.out.println("Reversed Number: " + rev);
    }
}
```

Output

```
Original Number: 1234
Reversed Number: 4321
```

Q2. Find Maximum of Three Numbers – Explain with Program

Finding the maximum of three numbers means identifying which number is the greatest among the three values.

Logic Used

You can compare three numbers using conditions:

If a > b and a > c → a is maximum
If b > a and b > c → b is maximum
Else → c is maximum

Java Program

```
public class MaxOfThree {
```

```
    public static void main(String[] args) {

        int a = 25, b = 42, c = 18;
        int max;

        if (a >= b && a >= c) {
            max = a;
        } else if (b >= a && b >= c) {
            max = b;
        } else {
            max = c;
        }

        System.out.println("Maximum Number: " + max);
    }
}
```

Output
Maximum Number: 42

Q3. Palindrome Check – Explain with Program
A palindrome number is a number that reads the same forward and backward.

Examples
121 → Palindrome
4554 → Palindrome
123 → Not a palindrome

Concept

1. Reverse the number using loop
2. Compare reversed number with original
3. If both are equal → Palindrome

Java Program

```
public class PalindromeCheck {
    public static void main(String[] args) {

        int num = 121;
        int temp = num;
        int rev = 0;

        while (temp != 0) {
```

```
            int digit = temp % 10;
            rev = rev * 10 + digit;
            temp /= 10;
        }

        if (num == rev) {
            System.out.println(num + " is a Palindrome Number.");
        } else {
            System.out.println(num + " is Not a Palindrome.");
        }
    }
}
```

Output
121 is a Palindrome Number.

Q4. Explain JVM / Bytecode

What is JVM?

JVM stands for Java Virtual Machine.
It is a runtime environment that executes Java programs.
JVM is platform-dependent (different JVMs for Windows, Linux, macOS).

Role of JVM

Loads Java class files
Verifies bytecode
Allocates memory
Executes instructions
Manages garbage collection
Provides security

Working Steps of JVM

1. Compilation:
Java compiler converts .java file → .class file (bytecode).
2. Class Loader:
Loads classes into JVM during program execution.
3. Bytecode Verifier:
Checks bytecode for any illegal code or security threats.
4. Interpreter & JIT Compiler:
Converts bytecode into machine code.

Interpreter: executes line-by-line
JIT (Just-in-Time) compiler: improves performance
5. Runtime Execution:
Actual execution of the program takes place.
6. Garbage Collector:
Automatically removes unused objects from memory.

What is Bytecode?

Bytecode is the intermediate code generated by the Java compiler.
It is platform-independent and can run on any OS using JVM.

Example bytecode format:

0: iconst_5
1: istore_1
2: getstatic
...

Why Bytecode?
Makes Java platform-independent
More secure (cannot run directly on OS)
Faster execution with JIT

Q5. String Method Demonstration – Explain with Examples
The String class in Java represents a sequence of characters.
It is immutable, meaning once created, it cannot be changed.

Below are the required methods:

1. charAt()
Returns the character at a specific index.

```
String s = "Java";
char ch = s.charAt(2);
System.out.println(ch);   // Output: v
```

2. contains()
Checks if the string contains a particular sequence.

```
String s = "Welcome to Java";
System.out.println(s.contains("Java"));  // true
```

3. format()

Used to format strings like printf.

```
String result = String.format("Name: %s, Age: %d", "Amit", 20);
System.out.println(result);
```

4. length()
Returns the length of the string.

```
String s = "Hello";
System.out.println(s.length());   // 5
```

5. split()
Breaks a string into parts based on a delimiter.

```
String s = "apple,banana,orange";
String[] arr = s.split(",");

for (String fruit : arr) {
    System.out.println(fruit);
}
```

Output

```
apple
banana
orange
```

Non-Repeated Questions:

Q1. Explain main() method in Java

The main() method is the entry point of every Java program.
It is the method where the execution of a Java application begins.
Without a proper main() method, the JVM cannot run the program, and it gives an error.

Syntax of main() Method

```
public static void main(String[] args)
```

This syntax has a special meaning:

1. public
The method must be accessible to JVM from anywhere.
JVM is outside the class, so the method must be public.

If it is not public, JVM cannot access it.

2. static
JVM does not create objects of the class to start execution.
To call a method without creating an object, it must be static.
Therefore, JVM directly calls main() using the class name.

3. void
The main method does NOT return any value.
JVM does not expect a return value.
That's why the return type is void.

4. main
It is the predefined name recognized by JVM.
Any other method name cannot be used as the starting point.

5. String[] args
This is used for command line arguments.
args is an array of String data, containing values passed from the command line.

Example usage:

java Test hello world

args[0] = "hello"
args[1] = "world"

Full Example

```
public class Demo {

    public static void main(String[] args) {
        System.out.println("Program Started");
    }

}
```

Why is main() important?

JVM starts program execution from main

Without it, the program cannot run

Acts as the controller for entire program flow

Q2. Discuss Scope of Variables

The scope of a variable means the region of the program where the variable is accessible.

In Java, there are four types of variable scopes:

🔵 1. Local Variables
Declared inside methods, constructors, or blocks
Accessible only within those blocks
Created when method is called, destroyed when method ends
Not stored in heap, stored in stack
Must be initialized before use

Example

```
void show() {
    int a = 10;  // local variable
    System.out.println(a);
}
```

🔵 2. Instance Variables (Non-static Variables)
Declared inside the class but outside any method
Belong to objects, not classes
Each object has its own copy
Stored in heap memory

Example

```
class Student {
    int id;  // instance variable
}
```

🔵 3. Static Variables (Class Variables)
Declared with the static keyword
Shared by all objects of the class
Only one copy exists in memory
Can be accessed using class name

Example

```
class College {
    static String collegeName = "ABC College";
```

```
}
```

🔵 4. Global Variables (Fields)
In Java, we call them fields.
Declared inside class but outside methods
Accessible throughout the class
Can be static or instance

Summary Table

Variable Type	Declared Where?	Accessible Where?
Local	Inside method/block/constructor	Only inside that method/block
Instance	Inside class (no static)	Through objects
Static	Inside class with static keyword	Using class name (shared by all objects)
Field (Global)	Inside class	Entire class

Q3. Explain Types of Arrays in Java
An array is a collection of elements of the same data type stored in continuous memory locations.
Arrays help store multiple values under a single variable name.

Java supports three types of arrays:

🔵 1. One-Dimensional Array
A linear collection of elements stored in sequential order.

Declaration

```
int[] arr = new int[5];
```

Initialization

```
arr[0] = 10;
arr[1] = 20;
```

Example

```
public class OneDArray {
    public static void main(String[] args) {
        int[] marks = {50, 60, 70, 80};

        for (int i : marks) {
            System.out.println(i);
```

```
        }
    }
}
```

🔵 2. Two-Dimensional Array
Represents matrix or table-like structure (rows and columns).

Declaration

```
int[][] matrix = new int[3][3];
```

Example

```
public class TwoDArray {
    public static void main(String[] args) {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(arr[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

🔵 3. Jagged Array (Array of Arrays)
A 2D array with variable number of columns in each row.

Example

```
public class JaggedArray {
    public static void main(String[] args) {
        int[][] arr = new int[3][];

        arr[0] = new int[2];
        arr[1] = new int[4];
        arr[2] = new int[3];

        System.out.println("Jagged Array Created");
    }
}
```

Advantages of Arrays

Easy to store multiple values
Fast access using index
Useful for mathematical operations and matrix programs

Limitations

Fixed size
Can store only same type of data

UNIT 2

FAQ(s):

Q1. Method Overloading – Definition, Need, and Example

Definition
Method Overloading is a feature of Object-Oriented Programming in Java that allows a class to have multiple methods with the same name but different parameters.
The difference may be in:

Number of parameters
Type of parameters
Order of parameters

It is also known as compile-time polymorphism because the decision of which method to call is made during compilation.

Why Method Overloading is Needed?

1. Improves code readability
Same operation can be performed with different data types or arguments.

2. Increases code reusability
You do not need to create multiple method names.

3. Makes the program easier to understand
For example, a calculator may have an add() method for both integers and floats.

Java Example: Method Overloading

class Calculator {

```
    // Method 1: Add two integers
    int add(int a, int b) {
        return a + b;
    }

    // Method 2: Add three integers
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Method 3: Add two double values
    double add(double a, double b) {
        return a + b;
    }
}

public class Test {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));         // 30
        System.out.println(c.add(10, 20, 30));     // 60
        System.out.println(c.add(5.5, 7.5));       // 13.0
    }
}
```

Conclusion

Method overloading makes Java programs flexible, readable, and maintainable by allowing the same method name for different operations.

Q2. Wrapper Classes and Their Use

What are Wrapper Classes?
Wrapper classes are predefined classes in Java that convert primitive data types into objects. Every primitive type has a corresponding wrapper class:

Primitive Type Wrapper Class

int Integer
float Float
double Double
char Character
boolean Boolean

byte Byte
short Short
long Long

Why Wrapper Classes Are Needed?

1. To use primitives in Collection Framework
Collections (ArrayList, HashMap, etc.) store only objects.

2. Autoboxing and Unboxing
Java automatically converts primitive to object and vice-versa.

3. Utility methods
Example: converting string to number using Integer.parseInt().

4. Immutability
Wrapper objects are immutable, ensuring safe data handling.

5. Working with Java Streams and Generics
Only objects are allowed in generics.

Autoboxing

Automatically converts primitive → object

```
int n = 10;
Integer obj = n;   // autoboxing
```

Unboxing

Object → primitive

```
Integer x = 20;
int y = x;   // unboxing
```

Example Program

```
public class WrapperDemo {
    public static void main(String[] args) {

        int a = 50;
        Integer obj = Integer.valueOf(a);   // Boxing
        int b = obj.intValue();             // Unboxing
```

```
        System.out.println("Wrapper Object: " + obj);
        System.out.println("Primitive Value: " + b);
    }
}
```

Q3. Use of this / static / final keywords

🔵 1. this Keyword
Definition
this is a reference variable in Java that refers to the current object.

Uses of this

1. To differentiate between instance variable and local variable
2. To call current class method
3. To call current class constructor
4. To return current object

Example

```
class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;          // using this keyword
        this.name = name;
    }

    void display() {
        System.out.println(id + " " + name);
    }
}
```

🔵 2. static Keyword
Definition
static is used to create class-level members that belong to the class, not objects.

Uses of static

1. Static variables → same for all objects
2. Static methods → can be accessed without object
3. Static blocks → run once when class loads
4. Used in memory management

Example

```
class College {
    static String collegeName = "ABC College";

    static void showCollege() {
        System.out.println(collegeName);
    }
}
```

🔵 3. final Keyword
Definition
final is used to restrict modifications.

It can be applied to:

Variables
Methods
Classes

Uses

1. final variable → constant value
2. final method → cannot be overridden
3. final class → cannot be inherited

Example

```
final class Vehicle {   // cannot be inherited
    final int speed = 120;   // constant

    final void display() {   // cannot be overridden
        System.out.println("Speed: " + speed);
    }
}
```

Q4. Create Class and Objects (Example: Student, Car, Calculator)
Definition of Class

A class is a blueprint or template that defines:

Data (variables)
Behavior (methods)

Definition of Object
An object is an instance of a class.

It has:

State
Behavior
Identity

🔵 Example 1: Student Class

```
class Student {
    int id;
    String name;

    void input(int i, String n) {
        id = i;
        name = n;
    }

    void display() {
        System.out.println("ID: " + id + ", Name: " + name);
    }

    public static void main(String[] args) {
        Student s1 = new Student();  // object creation
        s1.input(101, "Rahul");
        s1.display();
    }
}
```

🔵 Example 2: Car Class

```
class Car {
    String brand;
    int speed;

    void show() {
        System.out.println(brand + " running at " + speed + " km/hr");
    }

    public static void main(String[] args) {
        Car c = new Car();
        c.brand = "Honda";
```

```
        c.speed = 120;
        c.show();
    }
}
```

🔵 Example 3: Calculator Class

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int multiply(int a, int b) {
        return a * b;
    }

    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println("Sum: " + calc.add(10, 20));
        System.out.println("Product: " + calc.multiply(5, 6));
    }
}
```

Repeated Questions:

Q1. Constructor Overloading

Definition

Constructor Overloading in Java means creating multiple constructors with the same name (class name) but different parameter lists.

The difference can be:

Number of parameters
Type of parameters
Order of parameters

It helps in creating objects in different ways.

Why Constructor Overloading is Useful?

1. Flexibility

You can initialize objects with different levels of data.

2. Readability
Instead of writing multiple initialization methods, constructors are cleaner.
3. Automatic invocation
Constructor runs automatically when object is created.
4. Better object initialization
Each constructor provides a unique way of creating an object.

Example Program: Constructor Overloading

```
class Student {
    int id;
    String name;
    int age;

    // Constructor 1: No arguments
    Student() {
        id = 0;
        name = "Unknown";
        age = 0;
    }

    // Constructor 2: Two arguments
    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Constructor 3: Three arguments
    Student(int id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println(id + " " + name + " " + age);
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student(101, "Riya");
        Student s3 = new Student(102, "Aman", 20);

        s1.display();
```

```
        s2.display();
        s3.display();
    }
}
```

Q2. Wrapper Class Demonstration

Java wrapper classes convert primitive data types into objects.

They are used with collections, generics, utility methods and provide features like parsing, toString(), comparison, etc.

Primitive Types and Wrapper Classes

Primitive	Wrapper Class
int	Integer
float	Float
double	Double
char	Character
boolean	Boolean
byte	Byte
short	Short
long	Long

Autoboxing and Unboxing

Java automatically handles primitive ↔ object conversion.

Autoboxing (primitive → object)

```
int a = 50;
Integer obj = a;
```

Unboxing (object → primitive)

```
Integer x = 100;
int y = x;
```

Wrapper Class Demonstration Program

```
public class WrapperDemo {

    public static void main(String[] args) {

        // Primitive to Object (Autoboxing)
```

```
        int num = 10;
        Integer obj = num;

        // Object to Primitive (Unboxing)
        Integer val = new Integer(25);
        int data = val;

        // Parsing string to number
        String s = "123";
        int parsedValue = Integer.parseInt(s);

        System.out.println("Autoboxing result: " + obj);
        System.out.println("Unboxing result: " + data);
        System.out.println("Parsed integer: " + parsedValue);

        // Using wrapper class methods
        System.out.println("Max of 10 & 20: " + Integer.max(10, 20));
        System.out.println("Hex value of 15: " + Integer.toHexString(15));
    }
}
```

Q3. this Keyword Usage

The this keyword in Java is a reference variable that always refers to the current object.

Uses of this Keyword

1. To refer to current object's instance variables

Used when local and instance variables have same name.

Example:

```
class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;     // this refers to current object
        this.name = name;
    }
}
```

2. To call current class method

```
void display() {
```

```
    System.out.println("Display method called");
}

void show() {
    this.display();
}
```

3. To call current class constructor
Used for constructor chaining.

```
Employee() {
    this(101, "Default");
}

Employee(int id, String name) {
    this.id = id;
    this.name = name;
}
```

4. To return current object

```
class Test {
    Test getObject() {
        return this;
    }
}
```

5. To pass current object as an argument

```
class A {
    void display(B obj) {
        System.out.println("Object passed");
    }
}

class B {
    void show() {
        A a = new A();
        a.display(this);
    }
}
```

Example Program Showing all Uses of this

```
class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;        // differentiate instance vs local variable
        this.name = name;
    }

    Student() {
        this(101, "Unknown"); // calling another constructor
    }

    void show() {
        System.out.println(id + " " + name);
    }
}
```

Q4. Object Creation Type Programs

In Java, objects are created using the new keyword.

A class acts as a blueprint, and objects are real-world entities created from it.

Object Creation Syntax

```
ClassName obj = new ClassName();
```

Example 1: Student Object Creation

```
class Student {
    int roll;
    String name;

    void display() {
        System.out.println(roll + " " + name);
    }

    public static void main(String[] args) {
        Student s1 = new Student();  // object creation
        s1.roll = 1;
        s1.name = "Aman";
        s1.display();
    }
}
```

Example 2: Car Object Creation

```
class Car {
    String brand;
    int speed;

    void show() {
        System.out.println(brand + " runs at " + speed + "km/hr");
    }

    public static void main(String[] args) {
        Car c = new Car();
        c.brand = "BMW";
        c.speed = 200;
        c.show();
    }
}
```

Example 3: Calculator Object Creation

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        Calculator calc = new Calculator();  // object creation
        System.out.println("Sum: " + calc.add(10, 20));
    }
}
```

Example 4: Employee Object Creation Using Constructor

```
class Employee {
    int id;
    String name;

    Employee(int i, String n) {
        id = i;
        name = n;
    }

    public static void main(String[] args) {
        Employee e = new Employee(101, "Rahul");
```

```
        System.out.println(e.id + " " + e.name);
    }
}
```

Non-Repeated Questions:

Q1. How to Prevent Method Overriding?

In Java, method overriding happens when a subclass provides its own implementation of a method that is already defined in its superclass. While overriding is helpful for achieving runtime polymorphism, sometimes we want to stop subclasses from changing a specific method's behavior.
Java provides multiple ways to prevent method overriding:

(A) Using the final Keyword

The most common way is to declare the method as final.
A final method cannot be overridden.

Example:

```
class Parent {
    public final void show() {
        System.out.println("This is a final method. Cannot be overridden.");
    }
}

class Child extends Parent {
    // ERROR: Cannot override final method
    // public void show() { }
}
```

Explanation:
By marking show() as final, the Child class is forced to use the same implementation.

(B) Using a final Class

A final class cannot be inherited.
If there is no inheritance, there is no possibility of overriding.

Example:

```
final class Bank {
    void rules() {
```

```
        System.out.println("Bank rules cannot be changed.");
    }
}

// ERROR: Cannot inherit from a final class
// class MyBank extends Bank { }
```

(C) Making Methods private

Private methods are not visible to subclasses, so they cannot be overridden.

Example:

```
class Parent {
    private void data() {
        System.out.println("Private method");
    }
}
```

Private methods are not inherited, so overriding is impossible.

(D) Using static Methods

Static methods belong to the class, not the object.
They are not overridden; they are hidden.

```
class Parent {
    static void info() {
        System.out.println("Static method in Parent");
    }
}

class Child extends Parent {
    static void info() {  // This is method hiding, not overriding
        System.out.println("Static method in Child");
    }
}
```

Conclusion

To prevent method overriding, Java gives the following tools:

Method Can Override?Why?

final method ❌ No Method is locked
final class ❌ No Cannot extend class
private method ❌ No Not inherited
static method ❌ No Method belongs to class

Q2. Class vs Local Variables

Java variables are classified based on where they are declared and how long they exist. Two important types are Class variables and Local variables.

(A) Class Variables

A class variable is declared using the static keyword inside a class but outside any method. It is also called a static variable.

Key Features:

1. Shared by all objects of the class.
2. Memory is allocated once at class loading time.
3. Can be accessed using the class name.
4. Stored in the method area.
5. Initialized automatically (default values).

Example:

```
class Student {
    static String schoolName = "ABC School";  // Class variable
    String name;  // Instance variable

    void display() {
        System.out.println(name + " studies in " + schoolName);
    }
}
```

If we create 10 objects of Student, schoolName remains the same for all.

(B) Local Variables

Local variables are declared inside a method, constructor, or block.

Key Features:

1. Created when method is called and destroyed when method ends.
2. Not accessible outside the method.
3. Must be initialized manually (no default values).
4. Stored in the stack memory.

Example:

```
class Demo {
   void show() {
      int x = 10;  // Local variable
      System.out.println("Value: " + x);
   }
}
```

The variable x exists only during the show() method execution.

Comparison Table:

Feature	Class Variable	Local Variable
Declared	Inside class, outside methods	Inside method/block
Keyword	static	No keyword needed
Memory	Method area	Stack
Lifetime	As long as class is loaded	Only during method execution
Default value	Yes	No (must initialize)
Shared by objects	Yes	No
Access	ClassName.variable	Only inside the same method

UNIT 3

FAQ(s):

Q1. Types of Inheritance with Diagram

Definition
Inheritance is an OOP concept in Java where a new class (child/subclass) acquires the properties and methods of an existing class (parent/superclass).
It allows code reusability and hierarchical relationships.

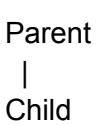
Types of Inheritance in Java

1. Single Inheritance
One subclass inherits from one superclass.

Syntax:

```
class Parent { }
class Child extends Parent { }
```

Diagram:

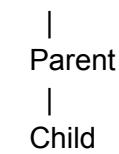


2. Multilevel Inheritance
A class inherits from a subclass, forming a chain of inheritance.

Syntax:

```
class Grandparent { }
class Parent extends Grandparent { }
class Child extends Parent { }
```

Diagram:



3. Hierarchical Inheritance
Multiple subclasses inherit from one superclass.

Syntax:

```
class Parent { }
class Child1 extends Parent { }
class Child2 extends Parent { }
```

Diagram:

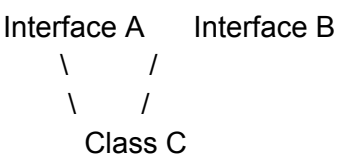


4. Multiple Inheritance (Using Interface)
A class implements more than one interface.
Java does not support multiple inheritance with classes to avoid ambiguity.

Syntax:

```
interface A { }
interface B { }
class C implements A, B { }
```

Diagram:



5. Hybrid Inheritance
Combination of two or more inheritance types.
Achieved using interfaces in Java.

Definition
An abstract class in Java is a class that cannot be instantiated directly.
It may contain:

Abstract methods (without body)
Concrete methods (with body)

Abstract classes are used when:

We want to provide a common base class
Force subclasses to implement specific methods

Syntax

```
abstract class Shape {
    abstract void area();   // abstract method
    void display() {        // concrete method
        System.out.println("This is a shape");
    }
}
```

Example Program

```
abstract class Shape {
    abstract void area();   // abstract method
```

```
    void display() {
        System.out.println("This is a shape");
    }
}

class Circle extends Shape {
    int radius;

    Circle(int r) {
        radius = r;
    }

    void area() {
        System.out.println("Circle area: " + (3.14 * radius * radius));
    }
}

public class TestShape {
    public static void main(String[] args) {
        Circle c = new Circle(5);
        c.display();  // inherited concrete method
        c.area();     // overridden abstract method
    }
}
```

Output

```
This is a shape
Circle area: 78.5
```

Q3. Multiple Inheritance Using Interface

Definition

Java does not support multiple inheritance with classes, but we can achieve it using interfaces. A class can implement more than one interface, combining their functionalities.

Example Program

```
interface A {
    void methodA();
}

interface B {
    void methodB();
}
```

```
class C implements A, B {    // Multiple inheritance using interfaces
    public void methodA() {
        System.out.println("Method A implemented");
    }

    public void methodB() {
        System.out.println("Method B implemented");
    }
}

public class Test {
    public static void main(String[] args) {
        C obj = new C();
        obj.methodA();
        obj.methodB();
    }
}
```

Output

```
Method A implemented
Method B implemented
```

Q4. Package Creation & Usage

Definition

A package is a collection of related classes and interfaces.

It helps in:

Organizing code

Avoiding name conflicts

Providing access control

Steps to Create and Use a Package

1. Create a Package

```
package mypackage;

public class Demo {
    public void show() {
        System.out.println("Hello from mypackage");
    }
}
```

2. Compile the Package

javac -d . Demo.java

This creates a folder named mypackage containing Demo.class.

3. Use the Package in Another Program

```
import mypackage.Demo;

public class TestPackage {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.show();
    }
}
```

4. Output

Hello from mypackage

Repeated Questions

Q1. toString(), equals(), finalize() Explanation

1. toString()
Returns string representation of an object
Default version prints class name + hashcode
Can be overridden for meaningful output

```
class Student {
    int id;
    String name;
    public String toString() {
        return id + " " + name;
    }
}
```

2. equals()
Compares content of objects
Default compares references only
Can be overridden to compare object properties

```
class Student {
    int id;
    String name;

    public boolean equals(Object obj) {
        Student s = (Student)obj;
        return this.id == s.id && this.name.equals(s.name);
    }
}
```

3. finalize()
Called by garbage collector before object is destroyed
Used to release resources

```
class Demo {
    protected void finalize() {
        System.out.println("Object destroyed");
    }
}
```

Q2. Hierarchical Inheritance Demonstration
Definition
Multiple subclasses inherit from a single superclass.
Example Program

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows");
    }
}

public class TestInheritance {
```

```
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();

        Cat c = new Cat();
        c.eat();
        c.meow();
    }
}
```

Output

```
Animal eats
Dog barks
Animal eats
Cat meows
```

Non-Repeated Questions

Q1. Visibility Controls in Java

Definition

Visibility control in Java defines how and where class members (variables, methods) can be accessed.

It is controlled by access modifiers.

Types of Access Modifiers

Modifier	Class	Package	Subclass	World
private	✔	✖	✖	✖
default	✔	✔	✖	✖
protected	✔	✔	✔	✖
public	✔	✔	✔	✔

1. private: Only accessible within the same class
2. default (no modifier): Accessible within the same package
3. protected: Accessible in package + subclass
4. public: Accessible everywhere

Q2. SavingsAccount Example Using Inheritance

Example Program

```
class Account {
    int accNo;
    double balance;

    Account(int accNo, double balance) {
        this.accNo = accNo;
        this.balance = balance;
    }

    void display() {
        System.out.println("Account No: " + accNo);
        System.out.println("Balance: " + balance);
    }
}

class SavingsAccount extends Account {
    double interestRate;

    SavingsAccount(int accNo, double balance, double interestRate) {
        super(accNo, balance);    // calling parent constructor
        this.interestRate = interestRate;
    }

    void displayInterest() {
        System.out.println("Interest Rate: " + interestRate + "%");
    }
}

public class TestAccount {
    public static void main(String[] args) {
        SavingsAccount sa = new SavingsAccount(101, 5000, 4.5);
        sa.display();             // parent method
        sa.displayInterest();     // child method
    }
}
```

Output

```
Account No: 101
Balance: 5000.0
Interest Rate: 4.5%
```

UNIT 4

Frequently Asked Questions

Q1. Life Cycle of a Thread

Definition
A thread is a lightweight process in Java.
The life cycle of a thread describes the various states a thread passes through from creation to termination.

States of a Thread

1. New (Created)
Thread is created but not yet started.
Example: Thread t = new Thread();

2. Runnable
After calling start(), the thread is ready to run.
JVM schedules it for execution.

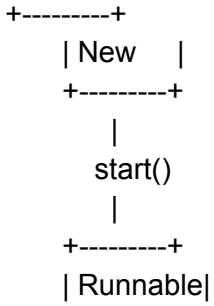
3. Running
Thread scheduler picks the thread and it executes its run() method.

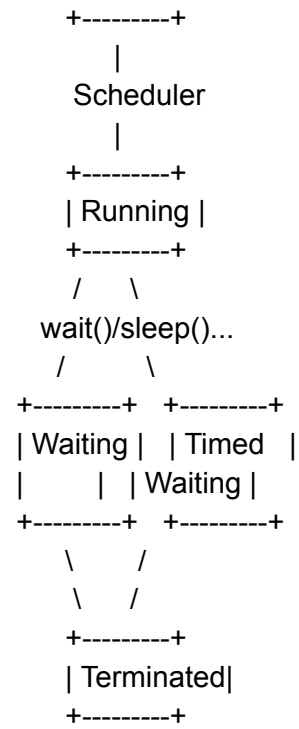
4. Waiting/Blocked
Thread waits for a resource or event.
It can be awakened by another thread (using notify()).

5. Timed Waiting
Thread waits for a specific time (using sleep(), join(timeout)).

6. Terminated/Dead
Thread completes execution or is stopped.
Cannot be restarted once terminated.

Thread Life Cycle Diagram





Thread Example

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}

public class TestThread {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();  // Moves thread from New → Runnable → Running
    }
}
```

Q2. Programs Involving Multiple Threads (Even-Odd, Timed Execution)

Example 1: Even and Odd Thread Printing

```
class Numbers {
    int counter = 1;
    int limit = 10;

    synchronized void printEven() throws InterruptedException {
        while (counter <= limit) {
```

```
            if (counter % 2 != 0) wait();
            System.out.println("Even: " + counter++);
            notify();
        }
    }

    synchronized void printOdd() throws InterruptedException {
        while (counter <= limit) {
            if (counter % 2 == 0) wait();
            System.out.println("Odd: " + counter++);
            notify();
        }
    }
}

class EvenThread extends Thread {
    Numbers n;
    EvenThread(Numbers n) { this.n = n; }
    public void run() {
        try { n.printEven(); } catch(Exception e) {}
    }
}

class OddThread extends Thread {
    Numbers n;
    OddThread(Numbers n) { this.n = n; }
    public void run() {
        try { n.printOdd(); } catch(Exception e) {}
    }
}

public class TestEvenOdd {
    public static void main(String[] args) {
        Numbers n = new Numbers();
        new EvenThread(n).start();
        new OddThread(n).start();
    }
}
```

Example 2: Timed Execution Using sleep()

```
class TimerThread extends Thread {
    public void run() {
        for(int i=1; i<=5; i++) {
```

```
            System.out.println("Timer: " + i + " seconds");
            try { Thread.sleep(1000); } catch(Exception e) {}
        }
    }
}

public class TestTimer {
    public static void main(String[] args) {
        TimerThread t = new TimerThread();
        t.start();
    }
}
```

Q3. try, catch, throw, throws

1. try

Block of code where exception may occur.

Syntax:

```
try {
    int x = 10 / 0;
}
```

2. catch

Handles the exception thrown in try block.

Syntax:

```
catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

3. throw

Used to explicitly throw an exception.

Syntax:

```
throw new ArithmeticException("Divide by zero error");
```

4. throws

Declares exceptions that a method may throw, letting the caller handle it.

Syntax:

```
void divide(int a, int b) throws ArithmeticException {
    System.out.println(a/b);
}
```

Example Program

```
public class ExceptionDemo {
    static void divide(int a, int b) throws ArithmeticException {
        if(b == 0) throw new ArithmeticException("Division by zero");
        System.out.println("Result: " + (a/b));
    }

    public static void main(String[] args) {
        try {
            divide(10, 0);  // may throw exception
        } catch(ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}
```

Repeated Questions

Q1. Banking Application with Exception

```
class Bank {
    int balance = 5000;

    void withdraw(int amount) throws Exception {
        if(amount > balance) throw new Exception("Insufficient Balance");
        balance -= amount;
        System.out.println("Withdrawal successful. Remaining: " + balance);
    }
}

public class TestBank {
    public static void main(String[] args) {
        Bank b = new Bank();
        try {
            b.withdraw(6000);
        } catch(Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
```

```
}
```

Q2. Even/Odd Thread Printing Programs

Already included under Q2 Example 1.

Non-Repeated Questions

Q1. Use of this Keyword Inside Thread Class
Explanation
In a thread class, this can be used to refer to the current thread object.
Useful when calling synchronized methods.
Can be passed as an argument to other methods.

Example

```
class MyThread extends Thread {
    public void run() {
        synchronized(this) {    // 'this' refers to current thread object
            for(int i=1; i<=5; i++) {
                System.out.println(Thread.currentThread().getName() + ": " + i);
            }
        }
    }
}

public class TestThisThread {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Output

```
Thread-0: 1
Thread-0: 2
```

UNIT 5

Frequently Asked Questions

Q1. Basics / Hierarchy of Stream Classes

Definition of Stream

In Java, a stream is a sequence of data elements that allows reading or writing data.
Streams abstract away the details of input/output devices and allow data to flow from source to destination.

Types of Streams in Java

1. Byte Streams (8-bit)
Handle raw binary data (images, audio, etc.)
Base classes: InputStream, OutputStream
Common subclasses: FileInputStream, FileOutputStream, BufferedInputStream, BufferedOutputStream

2. Character Streams (16-bit)
Handle text data (characters)
Base classes: Reader, Writer
Common subclasses: FileReader, FileWriter, BufferedReader, BufferedWriter

Hierarchy of Stream Classes

```
InputStream              Reader
     |                     |
  --------------------    -----------------
  FileInputStream         BufferedReader
  BufferedInputStream

OutputStream             Writer
     |                     |
  --------------------    -----------------
  FileOutputStream        BufferedWriter
```

Decorator Classes: BufferedInputStream, BufferedOutputStream, BufferedReader, BufferedWriter improve efficiency using buffers.

Advantages of Streams
Platform-independent I/O
Easy to read/write data
Can chain streams using decorators

Q2. File Read-Write Operations

1. Writing to a File

```
import java.io.*;

public class FileWriteExample {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("output.txt"); // Create file
            fw.write("Hello World!\n");
            fw.write("Java File Handling Example");
            fw.close();
            System.out.println("File written successfully");
        } catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

2. Reading from a File

```
import java.io.*;

public class FileReadExample {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("output.txt");
            int i;
            while ((i = fr.read()) != -1) {  // read character by character
                System.out.print((char)i);
            }
            fr.close();
        } catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Q3. ArrayList, LinkedList, HashSet, HashMap Programs

1. ArrayList Example

```
import java.util.ArrayList;

public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
```

```
        list.add("Banana");
        list.add("Orange");

        System.out.println("ArrayList Elements:");
        for(String fruit : list) {
            System.out.println(fruit);
        }
    }
}
```

2. LinkedList Example

```
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("Red");
        list.add("Blue");
        list.add("Green");

        System.out.println("LinkedList Elements:");
        for(String color : list) {
            System.out.println(color);
        }
    }
}
```

3. HashSet Example

```
import java.util.HashSet;

public class HashSetDemo {
    public static void main(String[] args) {
        HashSet<Integer> set = new HashSet<>();
        set.add(10);
        set.add(20);
        set.add(10); // duplicate ignored
        System.out.println("HashSet Elements: " + set);
    }
}
```

4. HashMap Example

```
import java.util.HashMap;

public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(101, "Riya");
        map.put(102, "Aman");
        map.put(103, "Rahul");

        System.out.println("HashMap Elements:");
        for(Integer key : map.keySet()) {
            System.out.println("ID: " + key + ", Name: " + map.get(key));
        }
    }
}
```

Repeated Questions

Q1. Create File & Append Data

```
import java.io.*;

public class AppendFileExample {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("data.txt", true); // true = append mode
            fw.write("Appending new line\n");
            fw.write("Another line added\n");
            fw.close();
            System.out.println("Data appended successfully");
        } catch(IOException e) {
            System.out.println(e);
        }
    }
}
```

Q2. HashMap Student Data Retrieval

```
import java.util.HashMap;

public class StudentHashMap {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();
```

```
        students.put(1, "Aman");
        students.put(2, "Riya");
        students.put(3, "Rahul");

        // Retrieve student with ID 2
        System.out.println("Student with ID 2: " + students.get(2));
    }
}
```

Q3. HashSet & ArrayList Programs

Already demonstrated in Q3.

Non-Repeated Questions

Q1. TreeSet Demonstration

```
import java.util.TreeSet;

public class TreeSetDemo {
    public static void main(String[] args) {
        TreeSet<Integer> set = new TreeSet<>();
        set.add(50);
        set.add(10);
        set.add(30);
        set.add(20);

        System.out.println("TreeSet Elements (Sorted & Unique): " + set);
    }
}
```

Output:

TreeSet Elements (Sorted & Unique): [10, 20, 30, 50]

Q2. Remove Specific Element from ArrayList

```
import java.util.ArrayList;

public class RemoveElementDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
```

```
        list.add("Banana");
        list.add("Orange");
        System.out.println("Original List: " + list);

        // Remove specific element
        list.remove("Banana");
        System.out.println("After Removing 'Banana': " + list);
    }
}
```

Output:

```
Original List: [Apple, Banana, Orange]
After Removing 'Banana': [Apple, Orange]
```